

Exercício 1: Escalonamento circular

Cenário 1 — Configuração de referência

Foram criados dois processos com escalonamento circular (Round-Robin): um CPU-bound (PID 145911) e um I/O-bound em disco (PID 152200), ambos com prioridade 0 (mesma prioridade), usando o quantum padrão do simulador. Rodou-se por 300s.

- Resultado: Temp UCP 174 (CPU-bound) vs 30 (I/O-bound); 87 trocas de contexto; Utilização UCP 60%.

Cenário 2 — Quantum aumentado

Os processos anteriores foram finalizados e recriados com os mesmos tipos e prioridades, mas com o valor de "Fatia de tempo" (quantum) aumentado na janela "Gerência do Processador". Rodou-se novamente por 300s.

- Resultado: Temp UCP 180 (CPU-bound) vs 14 (I/O-bound); apenas 31 trocas de contexto; Utilização UCP 80%.

Cenário 3 — Quantum diminuído

Os processos foram recriados mais uma vez, agora com o quantum reduzido em relação ao Cenário 1. Rodou-se pelo mesmo período de 300s para manter a comparação justa.

- Resultado: Temp UCP 103 (CPU-bound) vs 34 (I/O-bound); 137 trocas de contexto; Utilização UCP 60%.

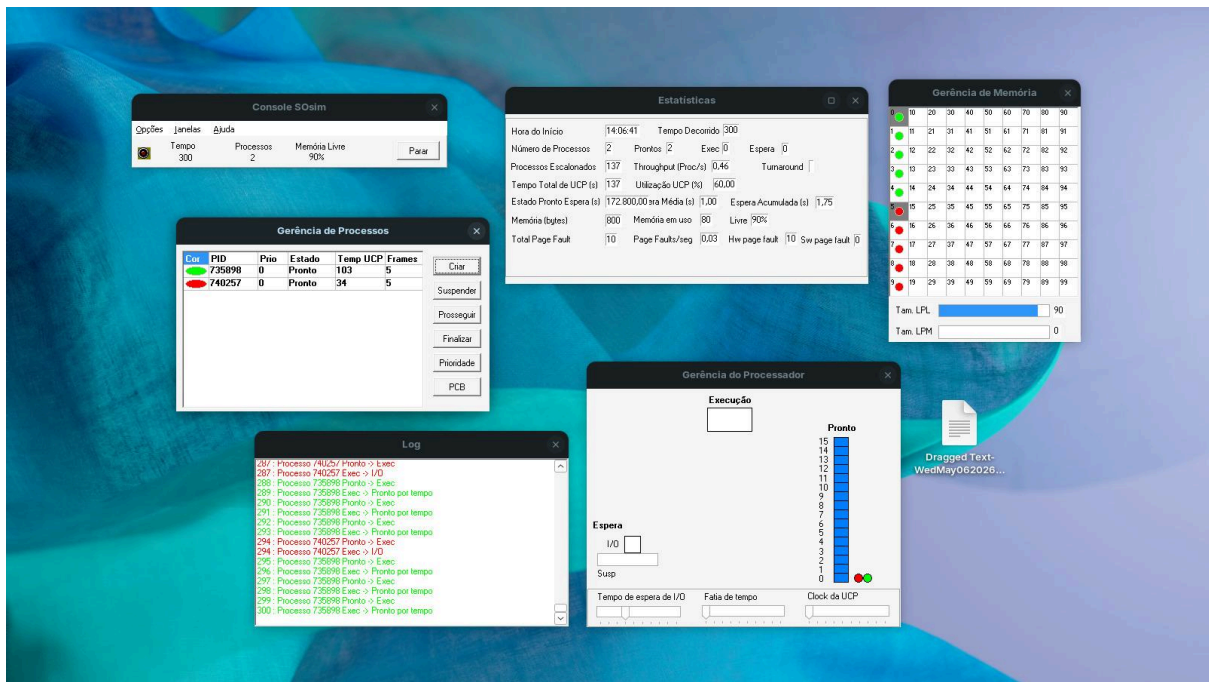
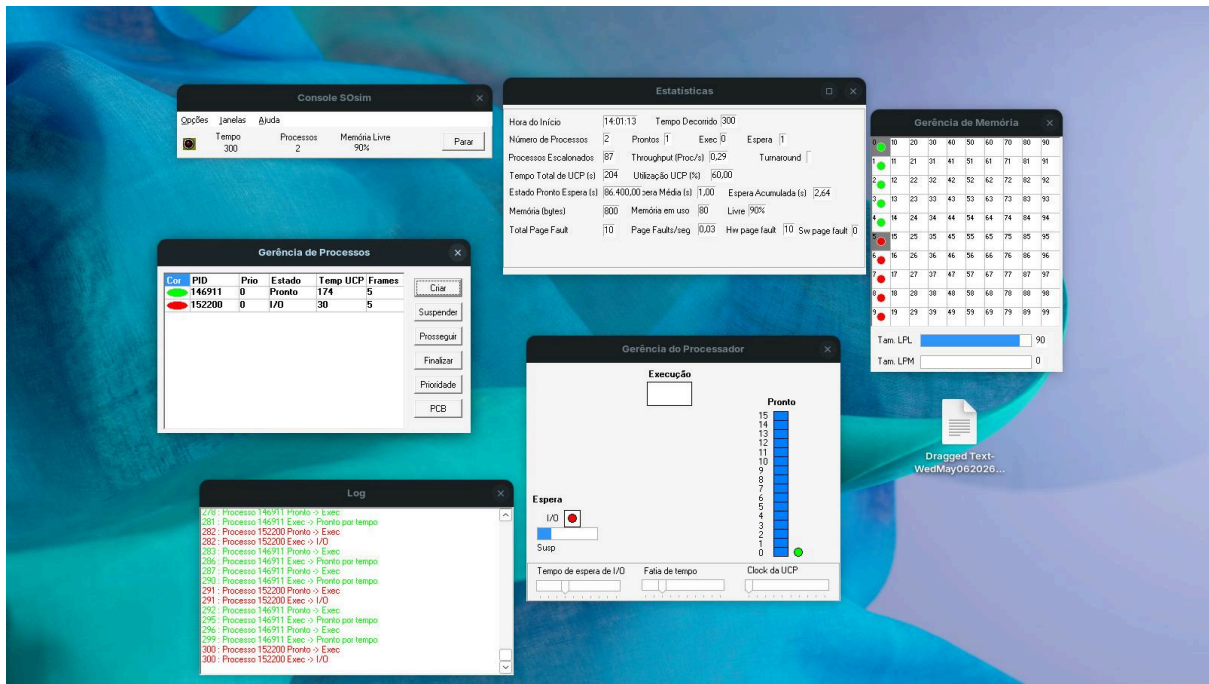
Conclusão

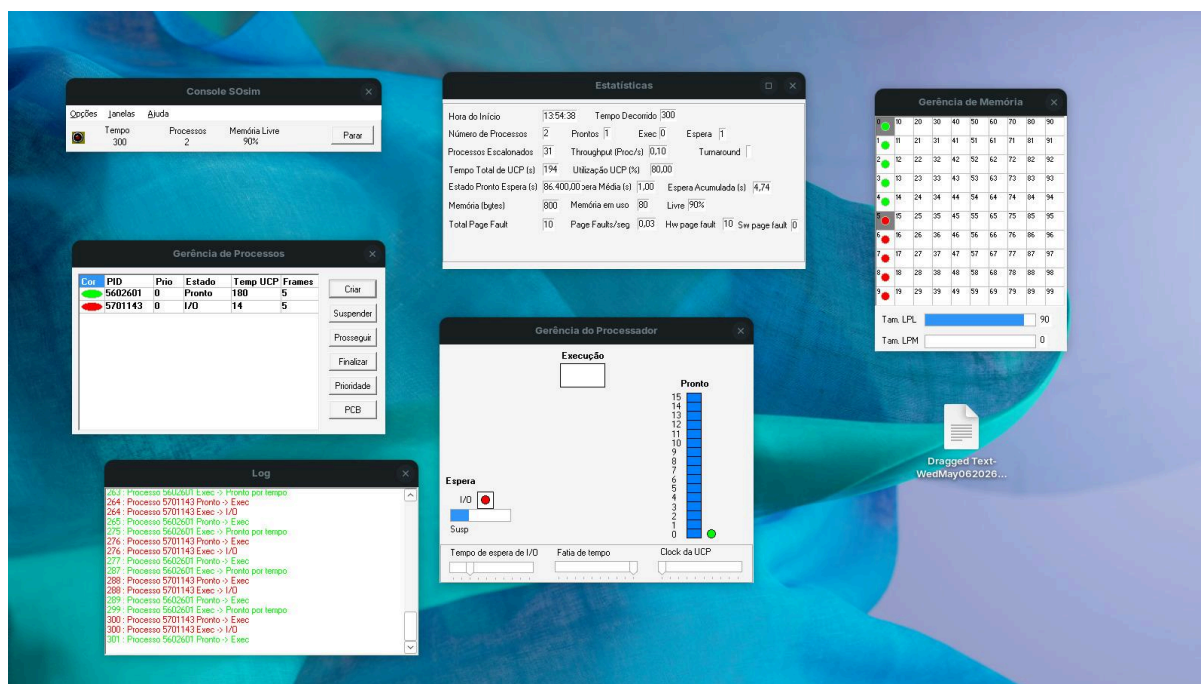
O dado-chave para entender o efeito do quantum é o número de "Processos Escalonados" (trocas de contexto): quanto maior o quantum, menos trocas ocorrem, e vice-versa.

- Com **quantum maior** (Cenário 2), a UCP fica mais tempo dedicada a cada processo por vez, gerando menos overhead de troca de contexto. Isso beneficiou fortemente o CPU-bound (180s de UCP, maior utilização geral: 80%), mas prejudicou o I/O-bound, que teve que esperar mais entre uma execução e outra (apenas 14s de UCP).
- Com **quantum menor** (Cenário 3), a UCP circula rapidamente entre os processos, dando ao I/O-bound mais chances de ser executado (34s, o melhor resultado dos três cenários para ele). Em compensação, o número de trocas de contexto disparou (137), o que aumenta o overhead do sistema e reduz o aproveitamento efetivo da UCP — refletido na queda da utilização total para 60%, mesmo tendo mais processos escalonados.

Ou seja: aumentar o quantum favorece processos CPU-bound e reduz overhead, mas piora o tempo de resposta de processos interativos (I/O-bound). Diminuir o quantum melhora a resposta dos processos I/O-bound, mas aumenta o custo de trocas de contexto e pode reduzir a eficiência geral do processador.

cenario 1, cenario 2 e cenario 3 respectivamente





Exercício 2 Escalonamento circular com prioridades

Cenário 1 — Tempo de espera de I/O padrão

Escalonamento configurado para "circular com prioridades estáticas", com processo CPU-bound (PID 2526790) em prioridade 3 e processo I/O-bound em disco (PID 2533025) em prioridade 4, mantendo o quantum e o tempo de espera de I/O no valor padrão. Rodou-se por 200s.

- Resultado: Temp UCP 95 (CPU-bound) vs 23 (I/O-bound); 46 trocas de contexto; Utilização UCP 100%.

Cenário 2 — Tempo de espera de I/O aumentado

Os processos foram finalizados e recriados com as mesmas prioridades (3 e 4), mas com o slider "Tempo de espera de I/O" deslocado para um valor maior — ou seja, o processo I/O-bound passou a ficar bloqueado por mais tempo a cada operação. Rodou-se novamente por 200s.

- Resultado: Temp UCP 114 (CPU-bound) vs 15 (I/O-bound); 43 trocas de contexto; Utilização UCP 80%.

Cenário 3 — Tempo de espera de I/O diminuído

Os processos foram recriados mais uma vez, com o tempo de espera de I/O reduzido em relação ao Cenário 1 (o I/O-bound volta mais rápido do bloqueio). Mesmo período de 200s.

- Resultado: Temp UCP 76 (CPU-bound) vs 36 (I/O-bound); 72 trocas de contexto; Utilização UCP 80%.

Comparação com o Exercício 1

No Exercício 1 (mesma prioridade para os dois processos), o CPU-bound dominava fortemente a UCP em qualquer cenário (ex: 174 vs 30 no cenário de referência). Aqui, mesmo o CPU-bound continuando à frente em tempo total, o I/O-bound se sai proporcionalmente melhor por ter prioridade maior (4 contra 3) — ele consegue disputar e ganhar a UCP sempre que fica pronto, o que não acontecia quando as prioridades eram iguais.

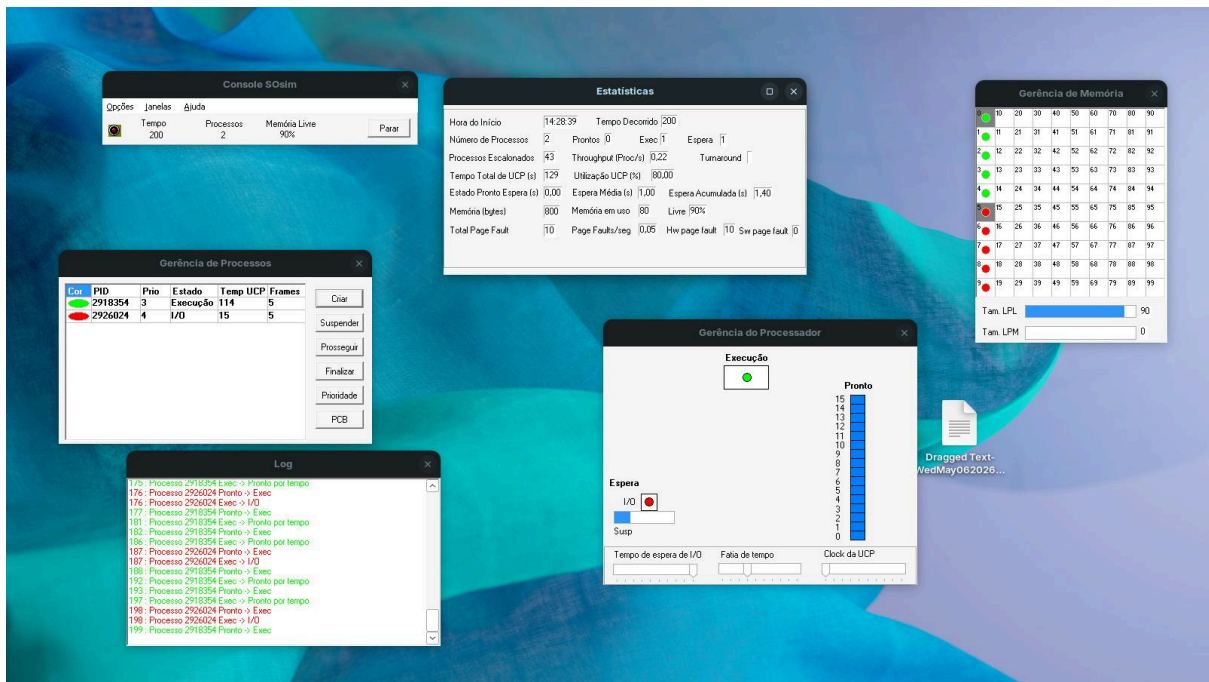
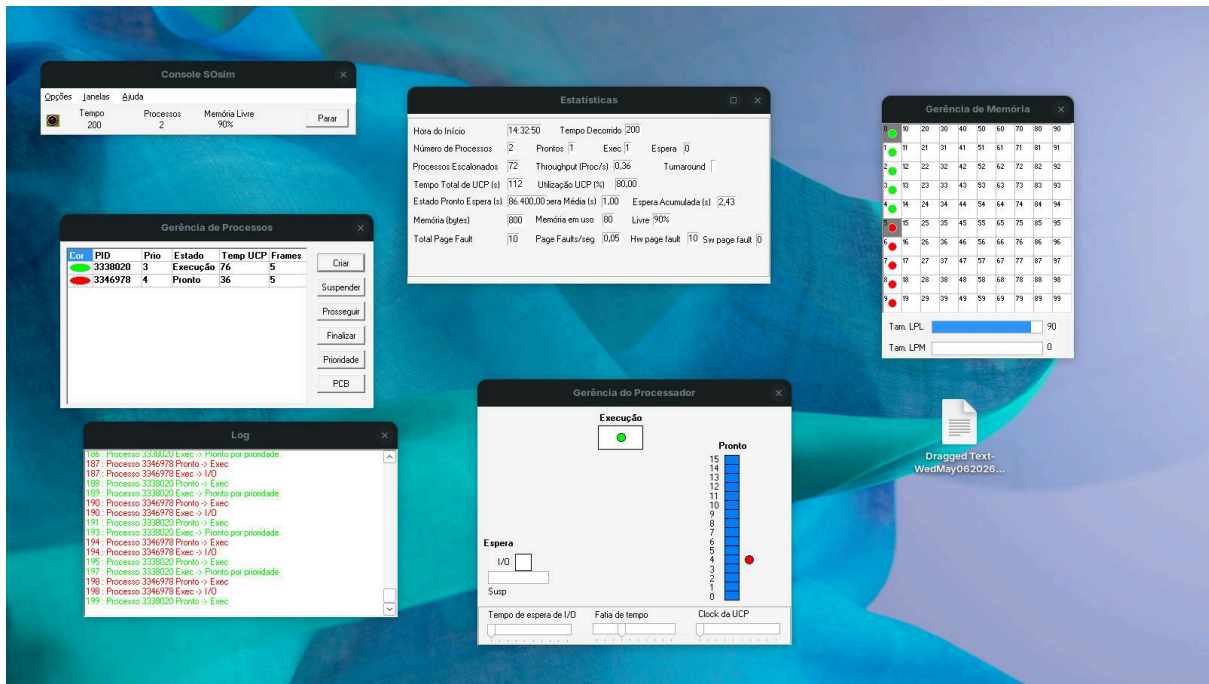
Conclusão — efeito do tempo de espera de I/O

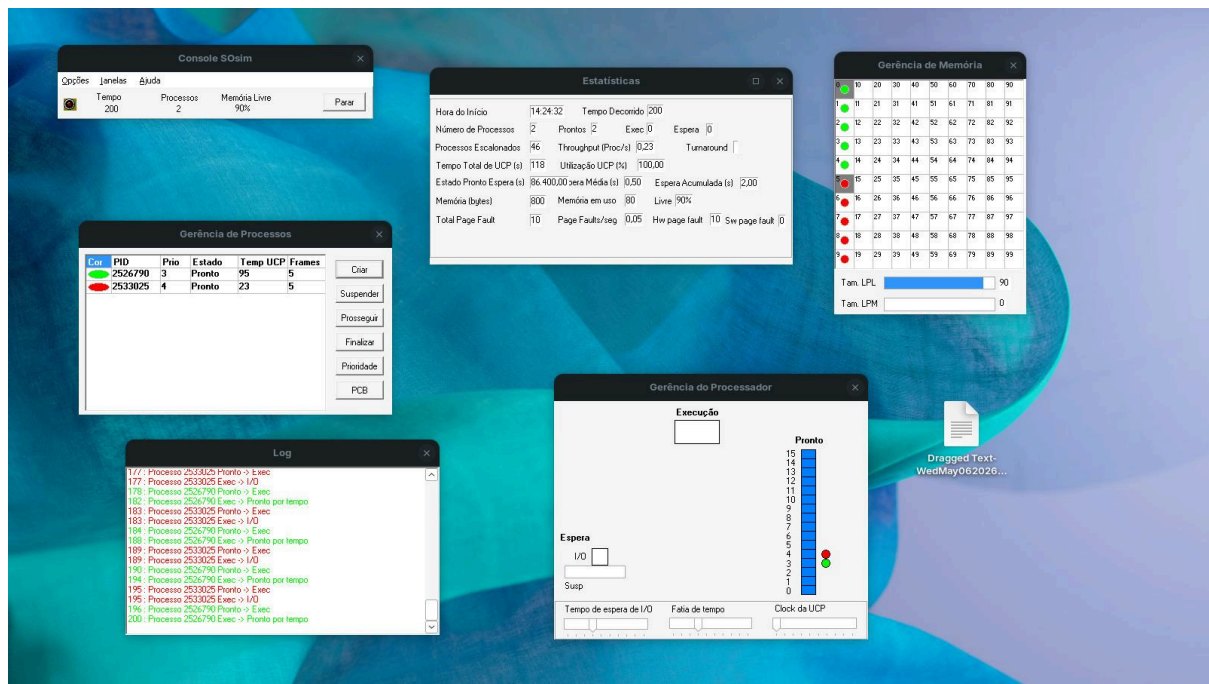
Os números confirmam a teoria:

- **Tempo de espera maior (Cenário 2):** o I/O-bound fica bloqueado por mais tempo a cada ciclo, então mesmo tendo prioridade mais alta, ele simplesmente não está disponível para disputar a UCP com frequência. Resultado: caiu para 15s de UCP (o pior resultado dos três), enquanto o CPU-bound aproveitou o tempo livre e subiu para 114s — o maior valor entre os cenários.
- **Tempo de espera menor (Cenário 3):** o I/O-bound volta rápido do bloqueio e, como tem prioridade maior, consegue preemptar/disputar a UCP com muito mais frequência. Isso se reflete no salto de trocas de contexto (72, quase o dobro do Cenário 1) e no melhor resultado do I/O-bound entre os três testes (36s de UCP), reduzindo a fatia do CPU-bound (76s).

Ou seja: aumentar o tempo de espera de I/O reduz a chance do processo I/O-bound de usar a UCP (mesmo com prioridade favorável), beneficiando o CPU-bound. Diminuir o tempo de espera faz o I/O-bound voltar rápido à fila e, combinado com sua prioridade maior, ele consegue uma fatia maior da UCP — à custa de mais trocas de contexto no sistema.

cenario 1, cenario 2 e cenario 3 respectivamente:





Exercício 3 Escalonamento circular com prioridades

Cenário 1 (padrão): Temp UCP 115 vs 0 — 29 trocas

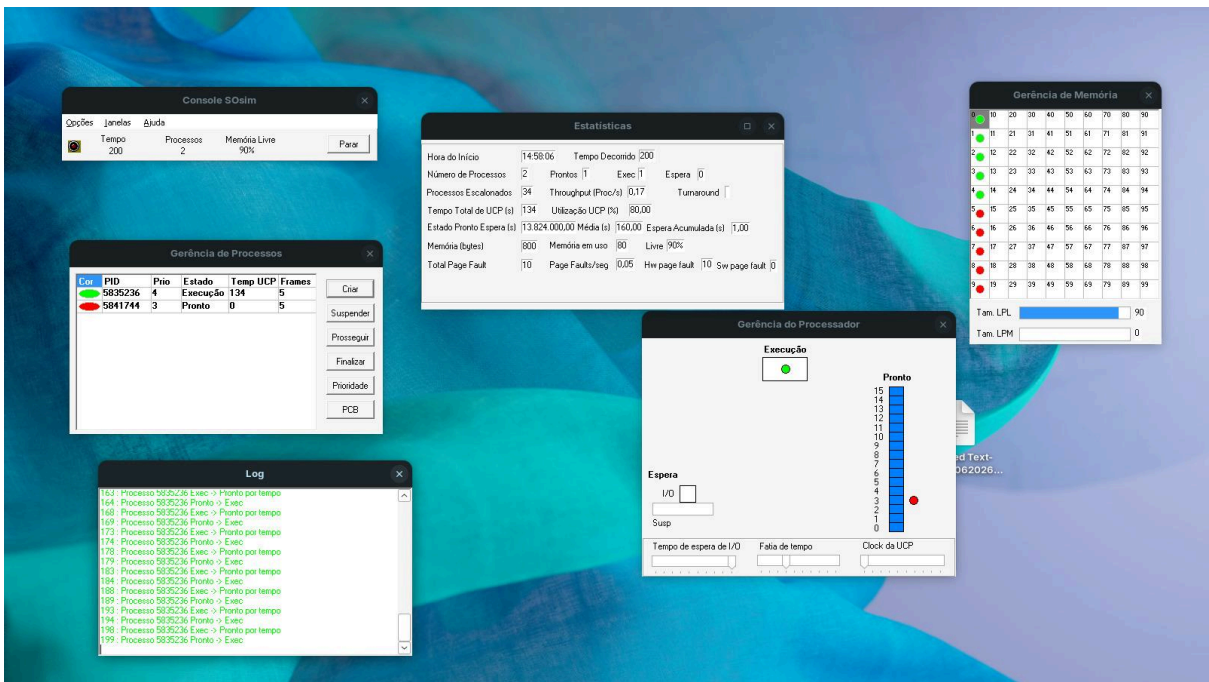
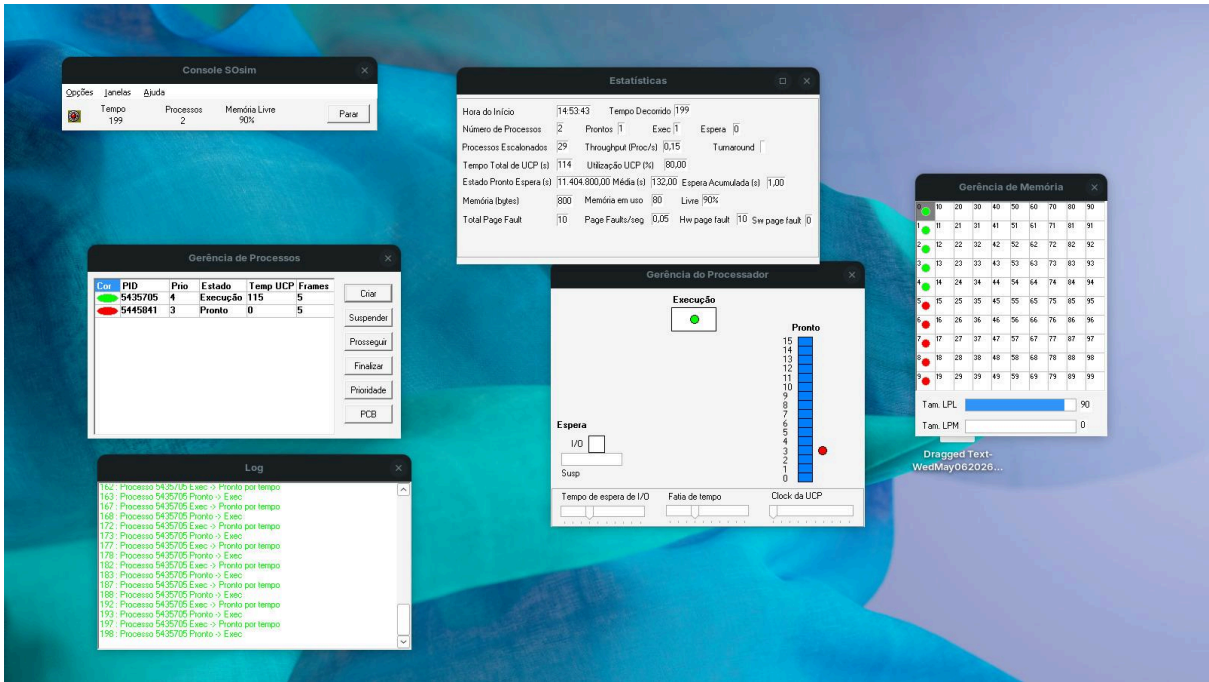
Cenário 2 (espera maior): Temp UCP 114 vs 0 — 68 trocas

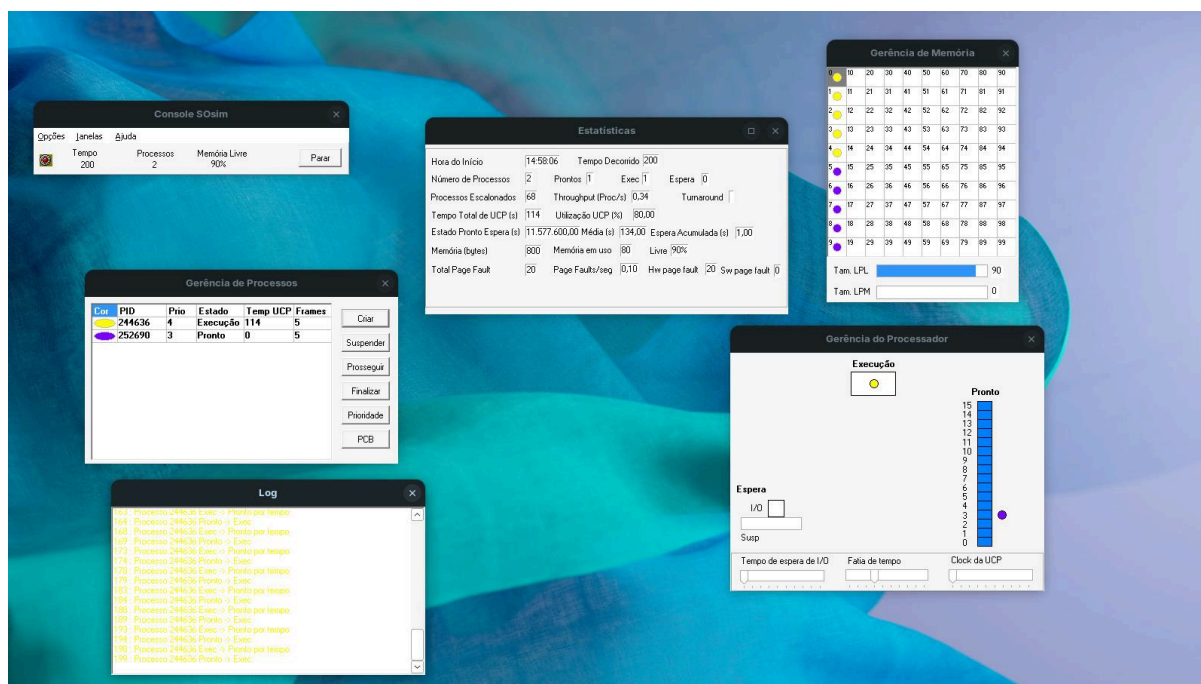
Cenário 3 (espera menor): Temp UCP 134 vs 0 — 34 trocas

Situação: Com CPU-bound em prioridade 4 e I/O-bound em prioridade 3, o I/O-bound não consegue **nenhum** segundo de UCP em nenhum dos três testes — o tempo de espera de I/O deixou de importar, porque a prioridade menor por si só já garante que ele nunca vença a disputa pela UCP. É um caso claro de **starvation** (inanição).

Critérios para definir prioridades:

- Favorecer processos I/O-bound/interativos, não CPU-bound (senão causa starvation, como visto aqui)
- Usar mecanismos de **aging** para evitar inanição (aumentar prioridade de quem espera muito)
- Considerar urgência da tarefa (tempo real vs comum)
- Considerar importância do processo para o sistema (kernel/serviços críticos)
- Equilibrar eficiência com justiça (fairness), garantindo que todo processo eventualmente rode





Exercicio 4 Escalonamento circular com prioridades (prioridade dinâmica – mecanismo adaptivo)

1. O que foi feito

Escalonamento circular com prioridade dinâmica, CPU-bound (PID 2037626) e I/O-bound em disco (PID 2045355) com a mesma prioridade base (3). Quantum e tempo de espera de I/O padrão. Rodado por 200s.

2. Resultados

- CPU-bound: 107s de UCP
- I/O-bound: 26s de UCP
- 52 trocas de contexto, 80% de utilização de UCP

O Log mostra várias entradas de "Aumento de prioridade +1" no I/O-bound — o sistema foi elevando sua prioridade automaticamente conforme ele esperava (aging).

3. Comparação com o Exercício 2

No Exercício 2, o I/O-bound só era favorecido porque alguém definiu manualmente uma prioridade maior (4 contra 3). Aqui os processos começaram iguais e o sistema ajustou a prioridade sozinho, chegando a um resultado equivalente (CPU 107 / I/O 26) sem nenhuma configuração manual.

4. Resposta à pergunta

A vantagem é que a prioridade dinâmica se adapta ao comportamento real de cada processo, em vez de depender de um valor fixo definido de antemão. Processos I/O-bound de perfis diferentes recebem reforços de prioridade proporcionais ao tempo de espera, evitando starvation automaticamente — sem exigir que o administrador acerte manualmente a prioridade ideal, como aconteceu (e falhou) no Exercício 3.

